



Reporters in Cypress

? What you'll learn

- How to use custom reporters in Cypress
- How to use multiple reporters
- How to merge reports across spec files

Introduction

After writing and running tests in Cypress, reviewing the results of those tests is crucial. Cypress provides several options to review results of a test run.

- [Cypress Cloud](#) - See test results, spec data, errors, screenshots, videos, and Test Replay in Cypress Cloud.
- Cypress App's open source built in and custom reporters.

This document covers how to use [built in](#) and [custom](#) reporters in Cypress App.

To get started with Cypress Cloud, [sign up](#) to start your **30 day free trial** - including all premium Cypress Cloud features and plenty of test results to let you experience the power of Cypress Cloud!

Built in reporters

Because Cypress is built on top of Mocha, that means any reporter built for Mocha can be used with Cypress. Here is a list of built in Mocha reporters.

- [Mocha's built-in reporters](#)

By default, Cypress uses the `spec` reporter to output information to `STDOUT`.



We've also added the two most common 3rd party reporters for Mocha. These are built into Cypress and you can use them without installing anything.

- `teamcity`
- `junit`

Custom reporters

Cypress supports creating your own custom reporters or using any kind of 3rd party reporter.

Installed locally

You can load [custom Mocha reporters](#) through a relative or absolute path. These can be specified in your Cypress configuration file or via the [command line](#).

For example, if you have the following directory structure:

```
> my-project
  > cypress
    > src
      > reporters
        - custom.js
```

You would specify the path to your custom reporter in either of the ways below.

Cypress configuration

cypress.config.js

cypress.config.ts

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  reporter: 'reporters/custom.js',
})
```

Command Line



```
npx cypress run --reporter reporters/custom.js
```

Installed via npm

When using custom reporters via npm, specify the package name.

Cypress configuration

cypress.config.js**cypress.config.ts**

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  reporter: 'mochawesome',
})
```

Command line

```
npx cypress run --reporter mochawesome
```

Reporter Options

Some reporters accept options that customize their behavior. These can be specified in your Cypress configuration or via [command line](#) options.

Reporter options differ depending on the reporter (and may not be supported at all). Refer to the documentation for the reporter you are using for details on which options are supported.

The below configuration will output the JUnit report to `STDOUT` and save it into an XML file.

Cypress configuration



cypress.config.js**cypress.config.ts**

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  reporter: 'junit',
  reporterOptions: {
    mochaFile: 'results/my-test-output.xml',
    toConsole: true,
  },
})
```

Command line

```
npx cypress run --reporter junit \
  --reporter-options "mochaFile=results/my-test-
  output.xml,toConsole=true"
```

Merging reports across spec files

Each spec file is processed completely separately during each `cypress run` execution. Thus each spec run *overwrites* the previous report file. To preserve unique reports for each specfile, use the `[hash]` in the `mochaFile` filename.

The following configuration will create separate XML files in the `results` folder. You can then merge the reported output in a separate step using a 3rd party tool. For example, for the [Mochawesome](#) reporter, you can use the [mochawesome-merge](#) tool.

Cypress configuration

cypress.config.js**cypress.config.ts**

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
```



```
reporter: 'junit',
reporterOptions: {
  mochaFile: 'results/my-test-output-[hash].xml',
},
})
```

Command line

```
npx cypress run --reporter junit \
  --reporter-options "mochaFile=results/my-test-output-
[hash].xml"
```

Multiple reporters

Oftentimes we see users wanting the ability to use multiple reporters. When running in CI, you might want to generate a report for `junit` and perhaps a `json` report. This is great, but by setting this reporter you won't receive any additional feedback while the tests are running!

The solution here is to use multiple reporters. You will have the benefit of both worlds.

We suggest using the npm module [cypress-multi-reporters](#).

Examples

Spec to `STDOUT`, save JUnit XML files

We want to output a `spec` report to `STDOUT`, while saving a JUnit XML file for each spec file.

We need to install additional dependencies:

- [cypress-multi-reporters](#): enables multiple reporters
- [mocha-junit-reporter](#) the actual junit reporter, as we cannot use the `junit` reporter that comes with Cypress



```
npm install cypress-multi-reporters mocha-junit-reporter --  
save-dev
```

Specify your reporter and reporterOptions in your Cypress configuration or via the [command line](#).

Cypress configuration

cypress.config.js

cypress.config.ts

```
const { defineConfig } = require('cypress')  
  
module.exports = defineConfig({  
  reporter: 'cypress-multi-reporters',  
  reporterOptions: {  
    configFile: 'reporter-config.json',  
  },  
})
```

Command line

```
npx cypress run --reporter cypress-multi-reporters \  
--reporter-options configFile=reporter-config.json
```

Then add the separate `reporter-config.json` file (defined in your configuration) to enable `spec` and `junit` reporters and direct the `junit` reporter to save separate XML files.

reporter-config.json

```
{  
  "reporterEnabled": "spec, mocha-junit-reporter",  
  "mochaJunitReporterReporterOptions": {  
    "mochaFile": "cypress/results/results-[hash].xml"  
  }  
}
```



```
}  
}
```

We recommend deleting all files from the `cypress/results` folder before running this command, since each run will output new XML files. For example, you can add the npm script commands below to your `package.json` then call `npm run report`.

package.json

```
{  
  "scripts": {  
    "delete:reports": "rm cypress/results/* || true",  
    "prereport": "npm run delete:reports",  
    "report": "cypress run --reporter cypress-multi-reporters -  
-reporter-options configFile=reporter-config.json"  
  }  
}
```

In case you want to combine generated XML files into a single one, [junit-report-merger](#) can be added. For example, to combine all files into `cypress/results/combined-report.xml` the `combine:reports` script can be added.

package.json

```
{  
  "scripts": {  
    "delete:reports": "rm cypress/results/* || true",  
    "combine:reports": "jrm cypress/results/combined-report.xml  
\"cypress/results/*.xml\"",  
    "prereport": "npm run delete:reports",  
    "report": "cypress run --reporter cypress-multi-reporters -  
-reporter-options configFile=reporter-config.json",  
    "postreport": "npm run combine:reports"  
  }  
}
```



Spec to `STDOUT`, produce a combined Mochawesome JSON file

This example is shown in the branch `spec-and-single-mochawesome-json` in <https://github.com/cypress-io/cypress-example-circleci-orb>. We want to output a "spec" report to `STDOUT`, save an individual Mochawesome JSON file per test file, and then combine all JSON reports into a single report.

We need to install some additional dependencies.

```
npm install mochawesome mochawesome-merge mochawesome-report-generator --save-dev
```

We need to configure the reporter in your [Cypress configuration](#) to skip the HTML report generation and save each individual JSON file in the `cypress/results` folder.

Cypress configuration

`cypress.config.js`

`cypress.config.ts`

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  reporter: 'mochawesome',
  reporterOptions: {
    reportDir: 'cypress/results',
    overwrite: false,
    html: false,
    json: true,
  },
})
```

Command line

```
npx cypress run --reporter mochawesome \
  --reporter-options
```




```
reportDir="cypress/results",overwrite=false,html=false,json=true
```

Our run will generate files `cypress/results/mochawesome.json`, `cypress/results/mochawesome_001.json`, ... Then we can combine them using the [mochawesome-merge](#) utility.

```
npx mochawesome-merge "cypress/results/*.json" >
mochawesome.json
```

We can now generate a combined HTML report from the `mochawesome.json` file using the <https://github.com/adamgruber/mochawesome-report-generator>:

```
npx marge mochawesome.json
```

It generates the beautiful standalone HTML report file `mochawesome-report/mochawesome.html` shown below. As you can see all test results, timing information, and even test bodies are included.



The screenshot shows the Cypress test runner interface. At the top, the title bar displays 'cypress-example-circleci-orb', a timer at '30.191s', and icons for 3 open files, 6 total files, 6 passed tests (green checkmarks), and 0 failed tests (red X's). Below the title bar, three spec blocks are shown, each with a summary row and a list of test results.

spec a
⌚ 10.56s 📄 2 ✓ 2

- ✓ works 48ms ⌚
`cy.wrap('foo').should('be.equal', 'foo');`
- ✓ waits a little 10.8s ⌚
`cy.wait(10000);`

spec b
⌚ 10.59s 📄 2 ✓ 2

- ✓ works 51ms ⌚
- ✓ waits a little 10.8s ⌚

spec c
⌚ 10.76s 📄 2 ✓ 2

- ✓ works 59ms ⌚
- ✓ waits a little 10.17s ⌚

©2019 Mochawesome was designed and built by Adam Gruber • v3.1.5

For more information, see [Integrating Mochawesome reporter with Cypress's](#)

History

Version	Changes
4.4.2	Custom Mocha reporters updated to use the version of Mocha bundled with Cypress. No need to install <code>mocha</code> separately to use custom reporters.

Last updated on **Jan 14, 2025**



CONTENTS

